

Lecture 5 Optimizing Program Performance

Performance Realities

In this chapter, we will explore how to make programs run faster via several different types of program optimization.

Optimizing Activities

Writing a efficient program requires several types of activities:

- Appropriate algorithms and data structures.
- Understand the capabilities and limitations of optimizing compilers.
- Divide a task into portions that can be computed in parallel.

We concentrate on 2 and 3. (1 is about data structure and algorithm)

Capabilities of Optimizing Compilers

Some programming languages are more easily optimized than others. Some features of C, such as the ability to perform pointer arithmetic and casting, make it challenging for a compiler to optimize.(need manually optimize the code)

Limitations of Optimizing Compilers

Sometimes the unoptimized version and optimized version has different behaviors.

For example,

```
void twiddle1(int *xp, int *yp)
{
    *xp += *yp;
    *xp += *yp;
}

void twiddle2(int *xp, int *yp)
{
    *xp += 2* *yp;
}
```

When `xp` and `yp` are equal, the result of `twiddle1` will be increased by a factor of 4 while `twiddle2` will be increased by a factor 3.

Optimization Blocker: Memory Aliasing

Aliasing:

- Two different memory references specify single location
- Easy to happen in C(allowed to do address arithmetic and direct access to storage structures)
- Get in habit of introducing local variables to avoid.

Expressing Program Performance

Cycles per element, abbreviated CPE, as a way to express program performance and help us understand the loop performance of an iterative program.

CPE

For example,

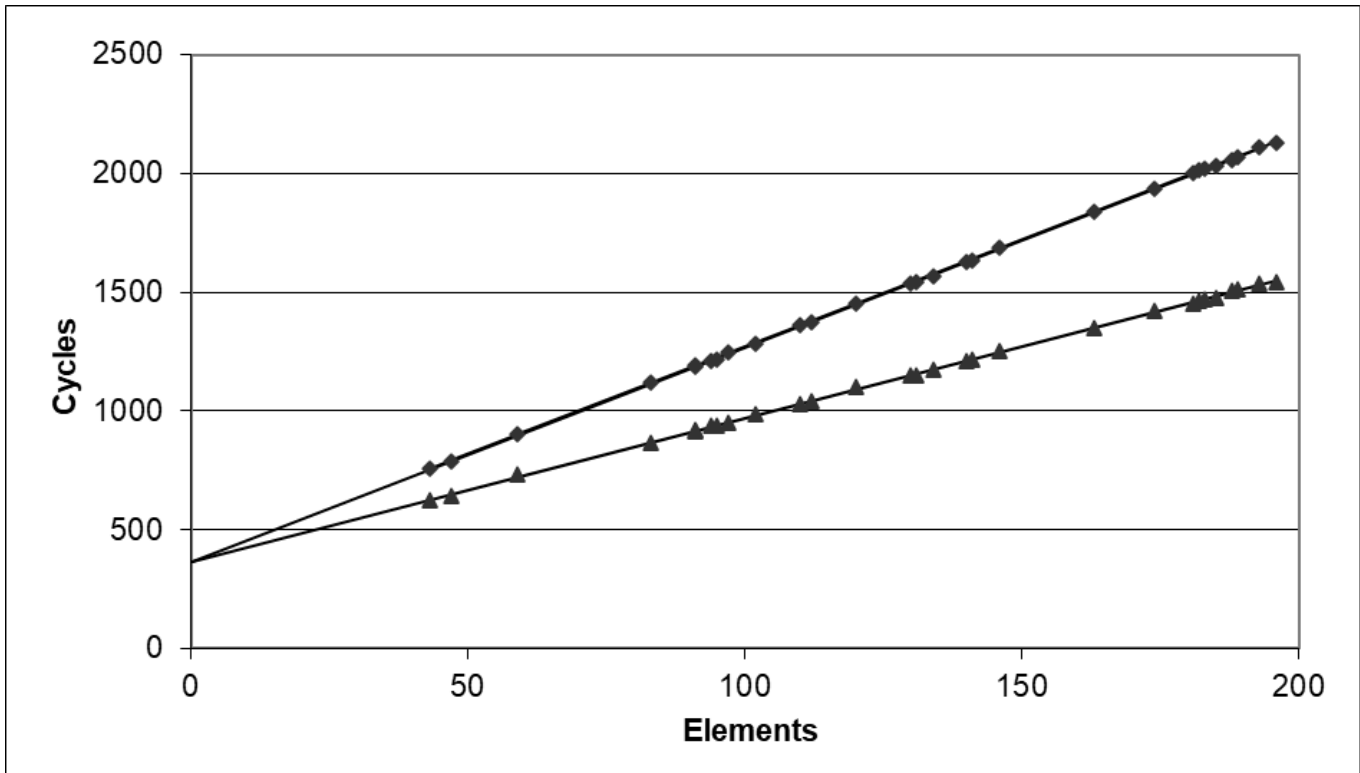
```

/* Compute prefix sum of vector a */
void psum1(float a[], float p[], long int n)
{
    long int i;
    p[0] = a[0];
    for (i = 1; i < n; i++)
        p[i] = p[i-1] + a[i];
}

void psum2(float a[], float p[], long int n)
{
    long int i;
    p[0] = a[0];
    for (i = 1; i < n-1; i+=2) {
        float mid_val = p[i-1] + a[i];
        p[i] = mid_val;
        p[i+1] = mid_val + a[i+1];
    }
    /* For odd n, finish remaining element */
    if (i < n)
        p[i] = p[i-1] + a[i];
}

```

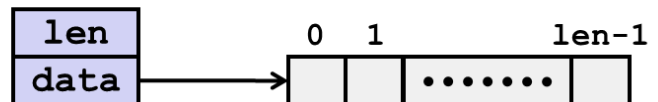
The function between cycles and elements can be characterized as $y = kx + b$, where k is CPE.



Benchmark Example: Data Type for Vectors

Basic Definitions:

```
/* data structure for vectors */
typedef struct{
    int len;
    data_t *data;
} vec;
```



```
/* retrieve vector element and store at val */
double get_vec_element(*vec, idx, data_t *val)
{
    if (idx < 0 || idx >= vec->len)
        return 0;
    *val = vec->data[idx];
    return 1;
}
```

```

void combine1(vec_ptr v, data_t *dest)
{
    long int i;
    *dest = IDENT;
    for (i = 0; i < vec_length(v); i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}

```

Eliminating Loop Inefficiencies

```

void combine2(vec_ptr v, data_t *dest)
{
    long int i;
    long int length = vec_length(v);
    *dest = IDENT;
    for (i = 0; i < length; i++) {
        data_t val;
        get_vec_element(v, i, &val);
        *dest = *dest OP val;
    }
}

```

Method	Integer		Double FP	
	Add	Mult	Add	Mult
Combine1	10.12	10.12	10.17	11.14
Combine2	7.02	9.03	9.02	11.03

Code motion reduces the computations that is performed multiple times but such that the result of the computation will not change, improving the efficiency of loop.

Reducing Procedure Calls

```

void combine3(vec_ptr v, data_t *dest)
{
    long int i;
    long int length = vec_length(v);
    data_t *data = get_vec_start(v);
    *dest = IDENT;
    for (i = 0; i < length; i++) {
        *dest = *dest OP data[i];
    }
}

```

Results in CPE (cycles per element)

Method	Integer		Double FP	
	Add	Mult	Add	Mult
Combine2	7.02	9.03	9.02	11.03
Combine3	7.17	9.02	9.02	11.03

This version moves the `get_vec_start()` from inner loop outward. No function calls in the inner loop and it access the array directly.

However, there is no apparent performance improvement and for integer sum even worse.

Since for each iteration, the value read from `dest` at the beginning and write to `dest` at the end, causing reading and writing wasteful.

Eliminating Unneeded Memory References

```

void combine4(vec_ptr v, data_t *dest)
{
    int i;
    int length = vec_length(v);
    data_t *d = get_vec_start(v);
    data_t acc = IDENT;
    for (i = 0; i < length; i++)
        acc = acc OP d[i];
    *dest = acc;
}

```

Method	Integer		Double FP	
Operation	Add	Mult	Add	Mult
Combine3	7.17	9.02	9.02	11.03
Combine4	1.27	3.01	3.01	5.01

Introduce a temporary variable `acc` in the loop to eliminate needless reading and writing of memory. Compared to `combine3`, we reduce the memory operations from two reads and one write to just a single read in each iteration.

Combine3 vs. Combine4

If we create an alias between the last element of the vector and the destination for storing the result. Then `combine3` will get wrong answer. So if operate memory directly:

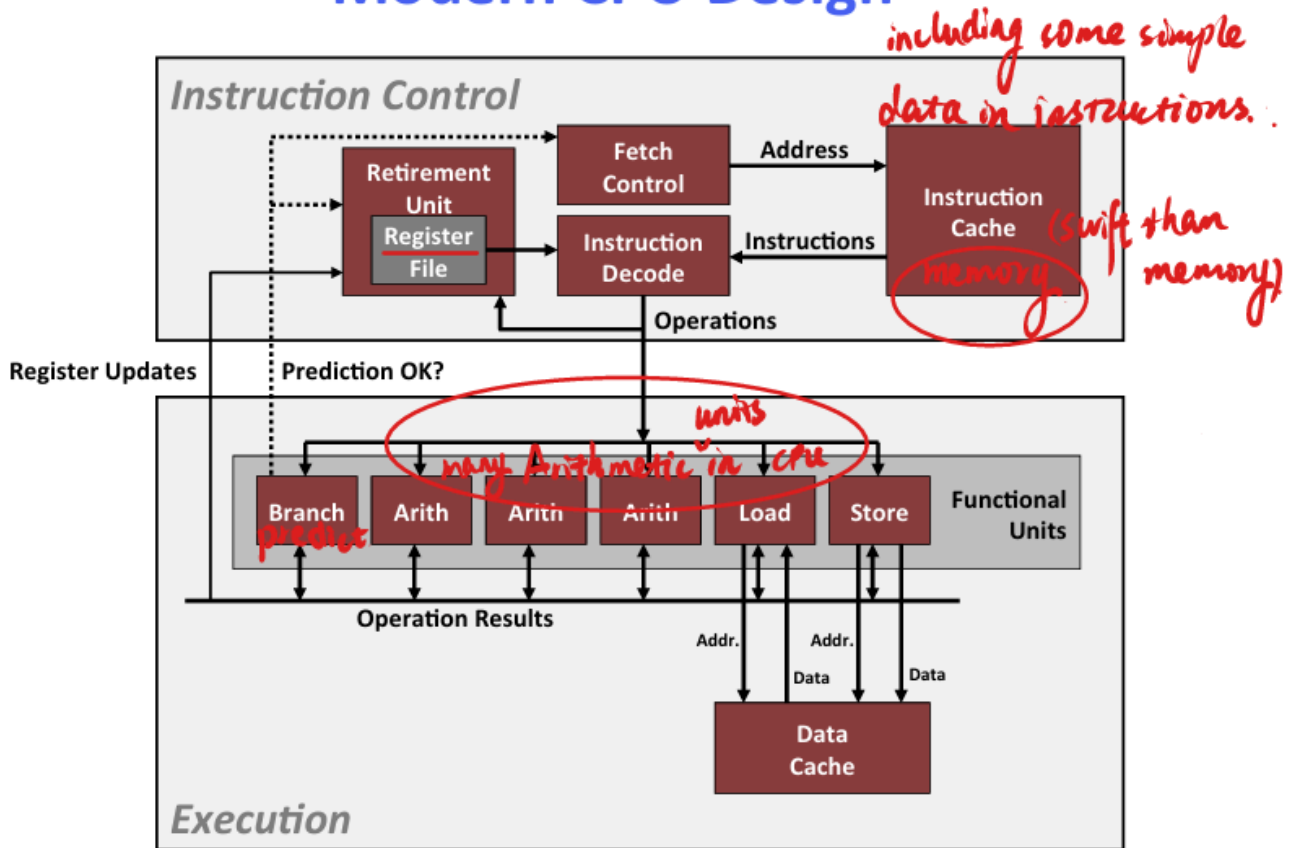
- With lower capacity
- Risk of memory alias

Classwork

Problem 5.3

Modern CPU Design

Modern CPU Design



Superscalar Processor

A superscalar processor can issue and execute multiple instructions in one cycle. The instructions are retrieved from a sequential instruction stream and are scheduled dynamically. So it can take advantage of instruction level parallelism.

Functional Unit Performance

- Latency: the total clock cycles required to perform the operation. (Fetch to PC renew of one operation)
- Issue time: the minimum number of clock cycles between two successive operations of the same type. (the first fetch to the second fetch, may parallelism so may less than latency)
- Capacity: the number of functional units capable of performing that operations.
- Throughput: the reciprocal of issue time. (the number of operations per clock cycle, derivative of capacity, operation/cycle)

Difference between latency and issue time, e.g.:

$a = b * c$ # 0-4 cycle for mul

$d = a + e$ # 5 cycle: need receive the result of mul, latency of mul is 5 cycles


```

a = b * c # 0 cycle
d = e * f # 1 cycle
# parallelism. Issue time 1 cycle

```

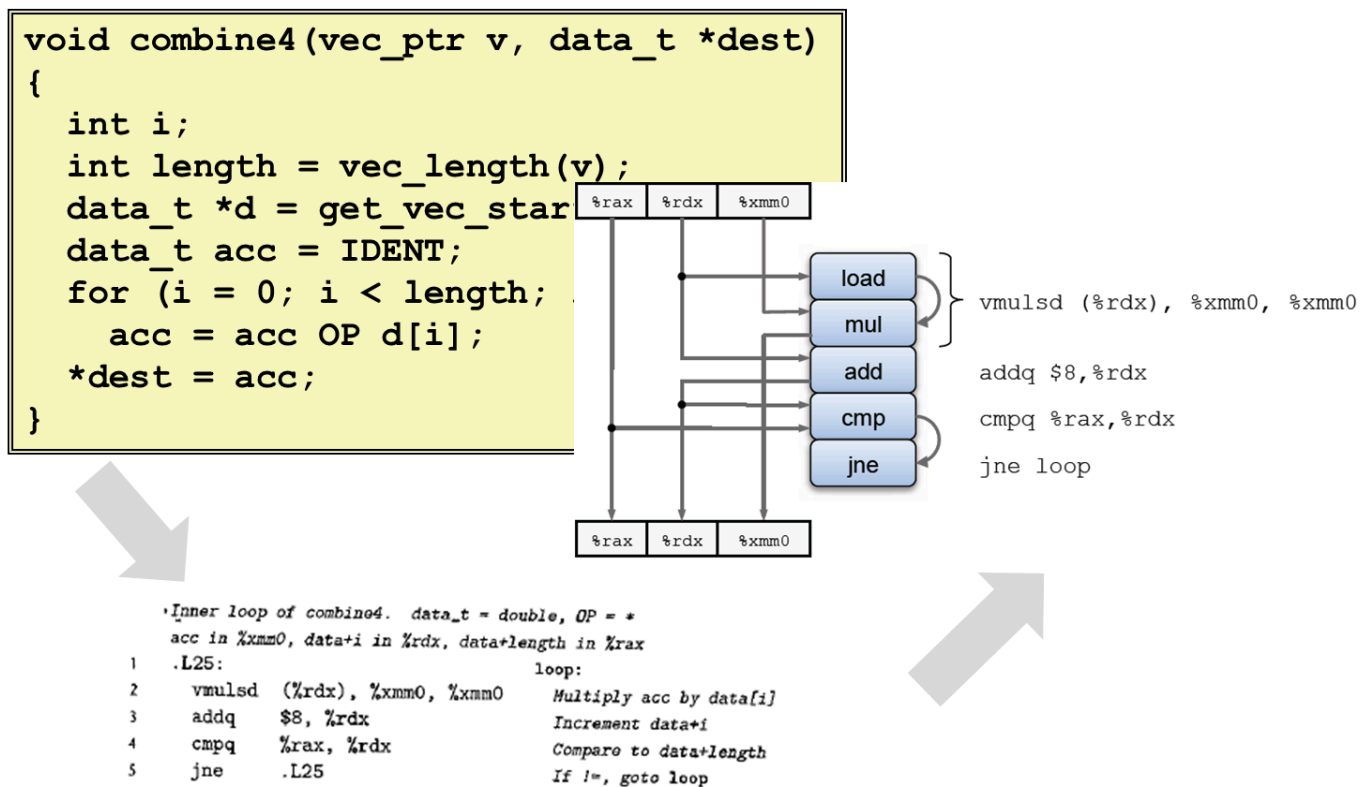
Two Fundamental Bounds on the CPE Values

- Latency bound: A minimum value for CPE for any function that must perform the combining operation in a strict sequence.(bounded by latency, sequential)
- Throughput bound: a minimum bound for CPE based on the maximum rate at which the functional units can produce results.(bounded by issue time, parallelism)

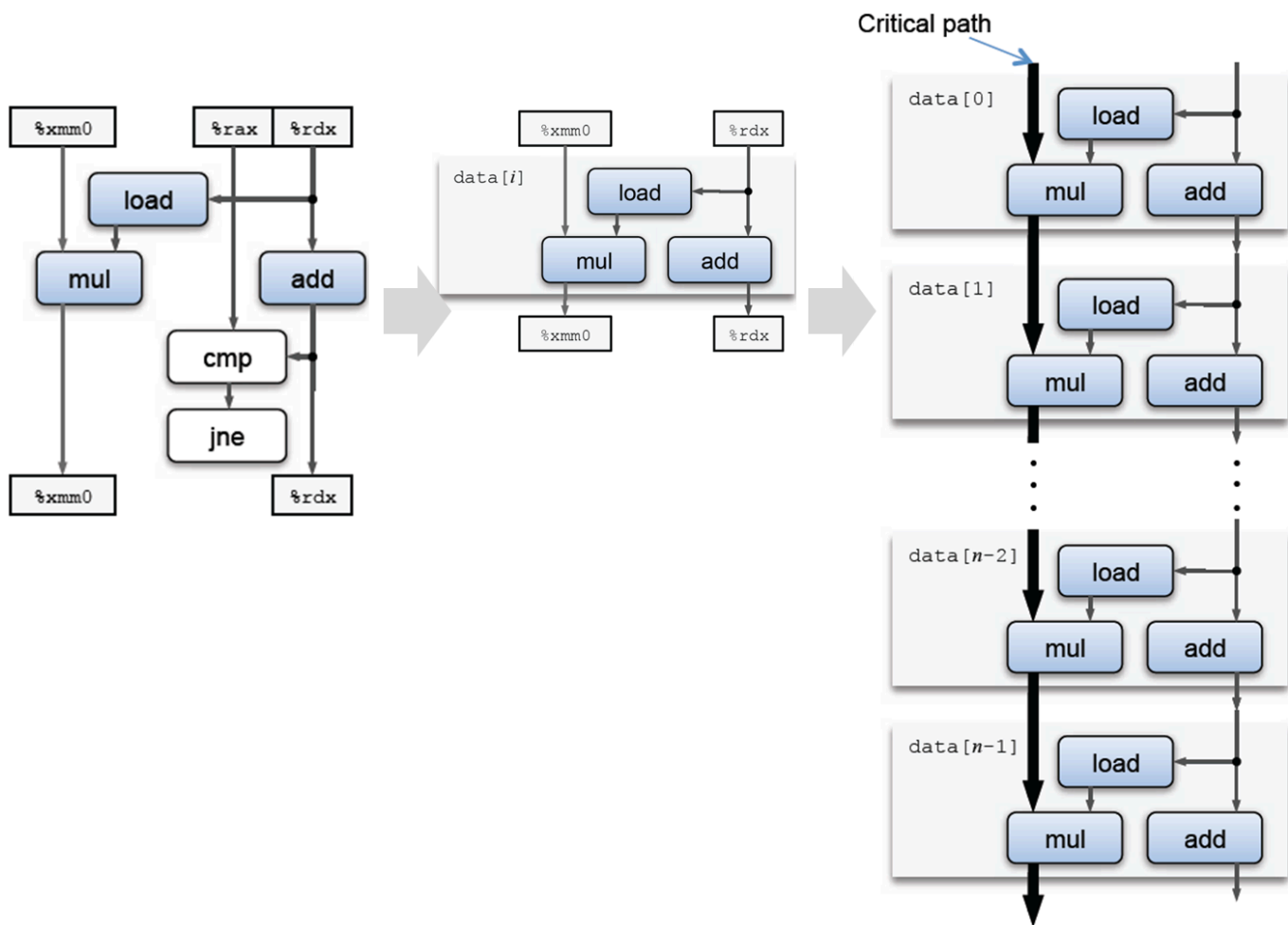
Abstract Model of Processor Operation

- Data-flow representation: a graphical notation showing how the data dependencies between the different operations constrain the order in which they are executed.
- Critical paths: a lower bound on the number of clock cycles required to execute a set of machine instructions(sequential)

From Machine-Level Code to Data-Flow Graphs



- Delete operations that not change registers.
- Delete registers that are not changed.



Loop Unrolling

```

/* Unroll loop by 2 */
void combine5(vec_ptr v, data_t *dest)
{
    long int i;
    long int length = vec_length(v);
    long int limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc = IDENT;

    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc = (acc OP data[i]) OP data[i+1];
    }

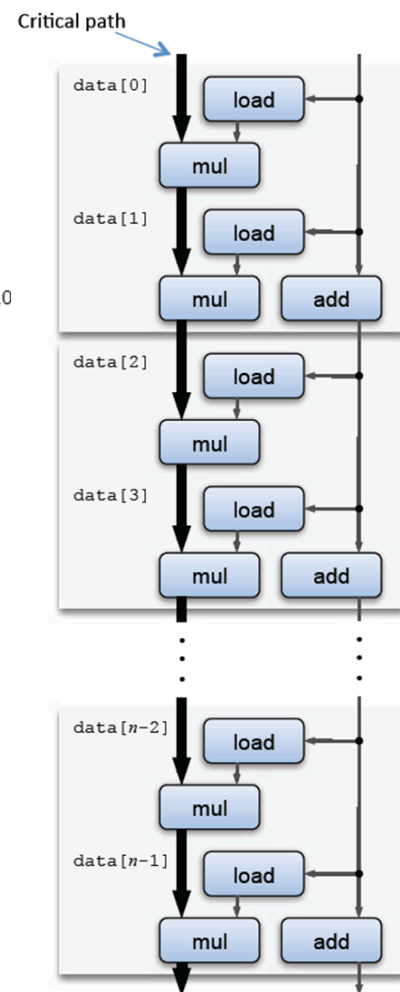
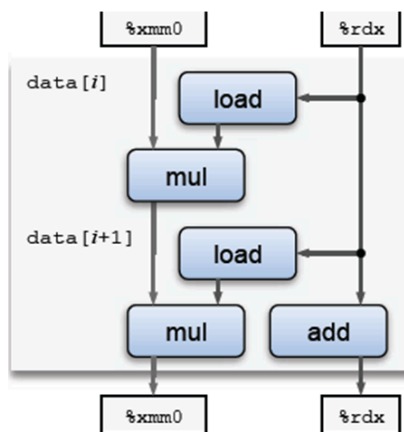
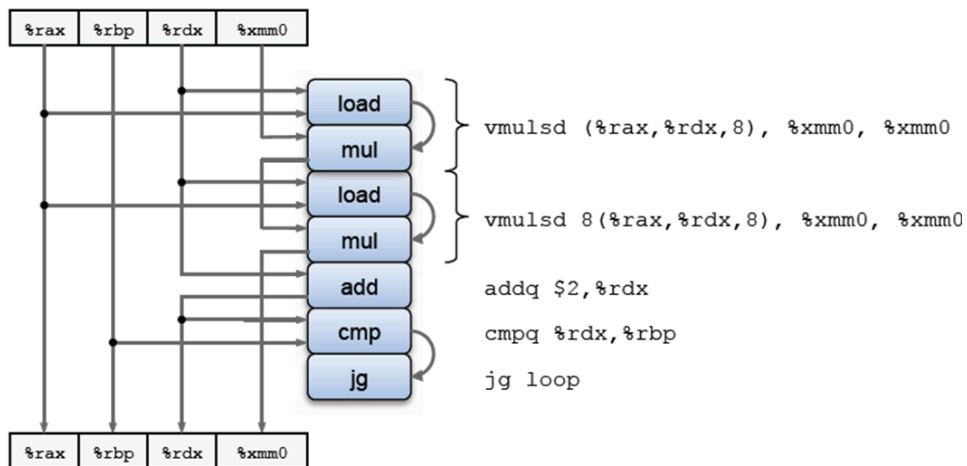
    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc = acc OP data[i];
    }
    *dest = acc;
}

```

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine4	515	No unrolling	1.27	3.01	3.01	5.01
combine5	532	2 × 1 unrolling	1.01	3.01	3.01	5.01
		3 × 1 unrolling	1.01	3.01	3.01	5.01
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

This version use **loop unrolling** to reduce the amount of iterations and opportunities for **prediction failure**.

And it reaches latency bound.



Enhancing Parallelism

```

/* Unroll loop by 2, 2-way parallelism */
void combine6(vec_ptr v, data_t *dest)
{
    long int i;
    long int length = vec_length(v);
    long int limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc0 = IDENT;
    data_t acc1 = IDENT;

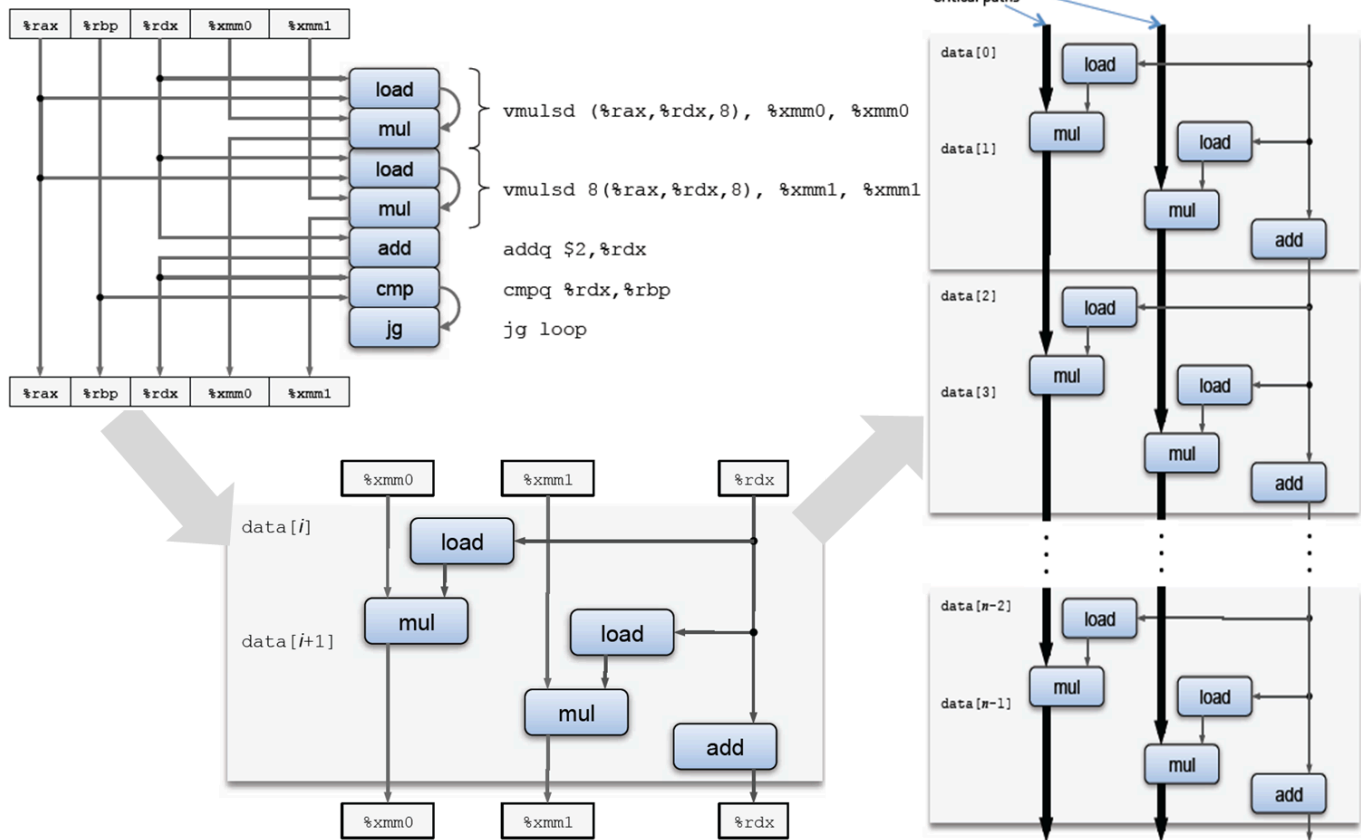
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc0 = acc0 OP data[i];
        acc1 = acc1 OP data[i+1];
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc0 = acc0 OP data[i];
    }
    *dest = acc0 OP acc1;
}

```

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine4	515	Accumulate in temporary	1.27	3.01	3.01	5.01
combine5	532	2×1 unrolling	1.01	3.01	3.01	5.01
combine6	537	2×2 unrolling	0.81	1.51	1.51	2.51
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

This version use loop unrolling by 2 and handle two accumulate in parallel for each iteration, breaking the latency bound.



Reassociation Transformation

```

/* Change associativity of combining operation */
void combine7(vec_ptr v, data_t *dest)
{
    long int i;
    long int length = vec_length(v);
    long int limit = length-1;
    data_t *data = get_vec_start(v);
    data_t acc = IDENT;

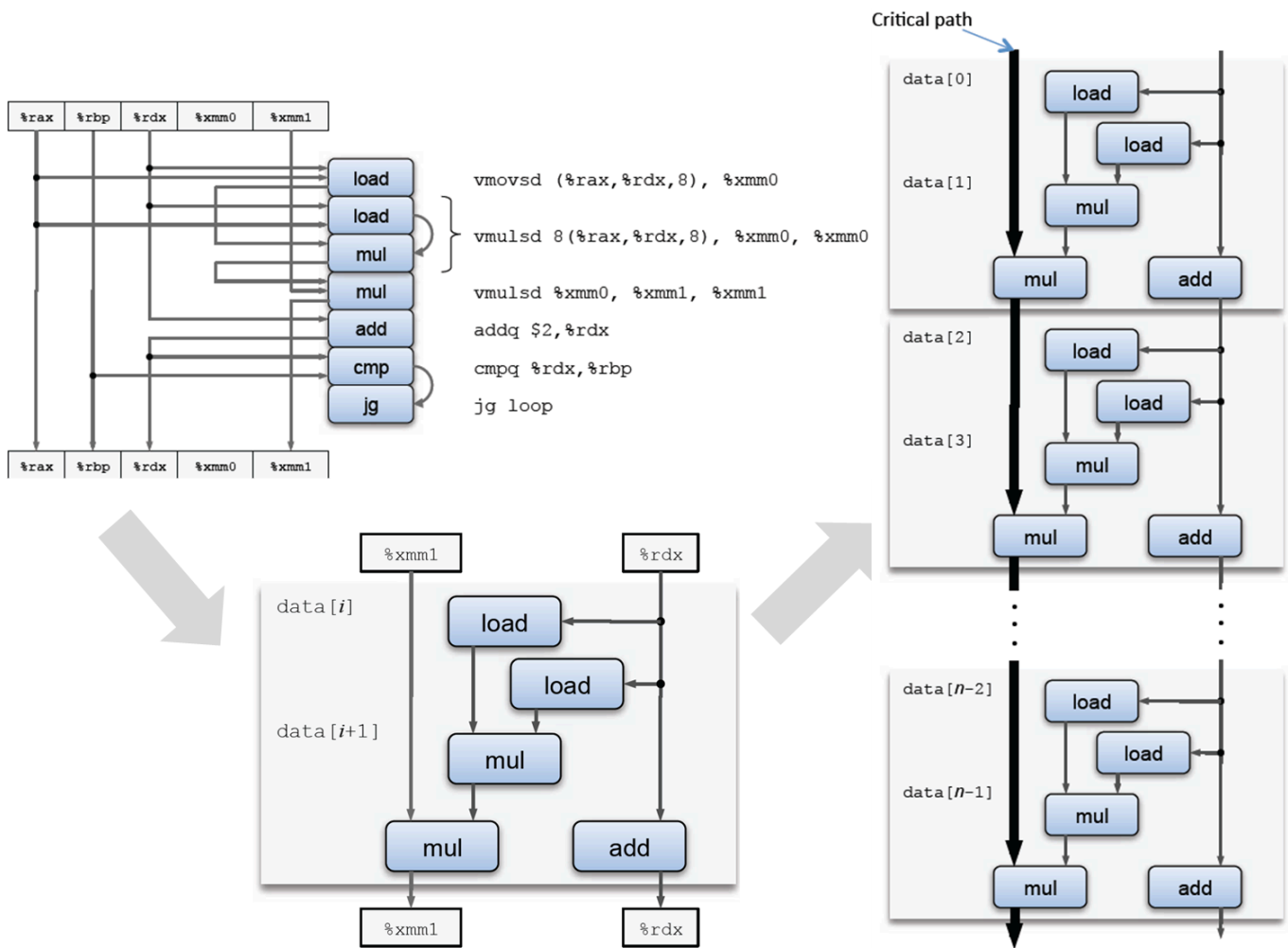
    /* Combine 2 elements at a time */
    for (i = 0; i < limit; i+=2) {
        acc = acc OP (data[i] OP data[i+1]);
    }

    /* Finish any remaining elements */
    for (; i < length; i++) {
        acc = acc OP data[i];
    }
    *dest = acc;
}

```

Function	Page	Method	Integer		Floating point	
			+	*	+	*
combine4	515	Accumulate in temporary	1.27	3.01	3.01	5.01
combine5	532	2 × 1 unrolling	1.01	3.01	3.01	5.01
combine6	537	2 × 2 unrolling	0.81	1.51	1.51	2.51
combine7	542	2 × 1a unrolling	1.01	1.51	1.51	2.51
Latency bound			1.00	3.00	3.00	5.00
Throughput bound			0.50	1.00	1.00	0.50

The version reassociates the calculation in the loop, removing use of extra register and parallelism universally for addition, reducing the length of critical path.



Meanwhile, the loop unrolling will reaches threshold when the factor reaches a bound.

Classwork

Problem 5.8, 5.15